

MongoDB: modello documentale, aggregation, indici e teorema CAP¹

Una sperimentazione su un replica set a 3 nodi

Approfondimento sui sistemi NoSQL

Big Data Analytics

Autore:

Manuel Testoni (219155)

12 giugno 2026

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



¹Code repository: https://github.com/ManuelTestoni/BigData/tree/main/Fitness_Exercise_Analysis — cartella Fitness_Exercise_Analysis

Indice

Abstract	2
1 Introduzione e contesto	2
2 Metodologia e setup sperimentale	2
3 RQ1 — Modellazione: dal CSV piatto ai documenti annidati	3
4 RQ2 — Aggregation framework: analytics nel database	4
5 RQ3 — Indici e performance	6
6 RQ4 — Il teorema CAP, sperimentato sul replica set	6
7 Conclusioni	8
Riferimenti bibliografici	9

Abstract

Questa relazione presenta un approfondimento sperimentale su MongoDB, il sistema NoSQL documentale più diffuso, condotto su un dataset reale di programmi di allenamento (Boostcamp, Kaggle): 605.033 righe in formato piatto e denormalizzato, con una struttura logica gerarchica a quattro livelli programma → settimana → giorno → esercizio. L'intera analisi gira contro un replica set a 3 nodi avviato con Docker Compose, che nella parte finale diventa anche il banco di prova per il teorema CAP. La sperimentazione è organizzata attorno a quattro research question: (RQ1) la convenienza di rimodellare il dataset piatto in documenti annidati (embedding vs referencing); (RQ2) l'uso dell'aggregation framework per fare analytics direttamente nel database; (RQ3) l'impatto degli indici sulle performance, misurato con `explain("executionStats")`; (RQ4) il comportamento del sistema sotto partizione di rete, osservato uccidendo dal vivo il nodo primario. I risultati principali: l'embedding riduce i dati logici del 87% (da 414 a 54 MB) e abbatta la latenza di lettura di due ordini di grandezza; gli indici fanno crollare i documenti esaminati da 605.033 a zero nel caso di covered query; il failover dal vivo mostra che MongoDB è un sistema CP, che sotto partizione sacrifica la disponibilità in scrittura per preservare la consistenza. Tutto il codice e le istruzioni di riproduzione sono pubblicati su un repository GitHub corredato di README.

1 Introduzione e contesto

Per l'approfondimento sui sistemi NoSQL ho scelto MongoDB, un database citato a lezione che si basa su dati semi strutturati. Il modello utilizzato da MongoDB rinuncia allo schema rigido, tipico degli RDBMS, in favore di documenti BSON annidati [1]. In questo approfondimento andremo a capire quando questo modello conviene davvero, quali garanzie offre e a cosa rinuncia, dato che sappiamo dal teorema CAP che non si possono garantire tutti e tre i principi.

Il dataset. Come caso di studio uso un dataset Kaggle di programmi di allenamento della piattaforma Boostcamp [4], composto da 605.033 righe, una per esercizio. Il CSV è completamente denormalizzato, quindi ogni riga ripete tutti i metadati del programma (titolo, descrizione, livello, obiettivo, equipment, ...). I dati però hanno una gerarchia naturale a quattro livelli, programma → settimana → giorno → esercizio, ed è proprio questo che mi ha spinto a voler utilizzare un'architettura NoSQL documentale. Per la simulazione d'analisi ho virtualizzato 3 nodi MongoDB su container Docker.

Le research question. Le domande che mi sono posto sono quattro, ciascuna sperimentata e misurata sul replica set:

- RQ1 — Modellazione: conviene rimodellare il dataset piatto in documenti annidati o possiamo lasciarli standard? Confronto spazio e accesso.
- RQ2 — Aggregation framework: che analisi si possono fare direttamente nel database con le pipeline di aggregazione, senza estrarre i dati?
- RQ3 — Indici: che impatto possono avere gli indici supportati da MongoDB? Anche in questo assignment profilo le query.
- RQ4 — Teorema CAP: come si comporta il replica set sotto partizione? Vediamo scenari in cui i nodi falliscono.

La Sezione 2 descrive l'infrastruttura sperimentale; le Sezioni 3–6 rispondono alle quattro RQ con i risultati ottenuti; la Sezione 7 riassume cosa è stato provato e la sua rilevanza.

2 Metodologia e setup sperimentale

L'intera sperimentazione è riproducibile da un notebook Jupyter (`fitness_exercise_analysis.ipynb`) che si connette, come sorgente unica, a un replica set `rs0` a 3 nodi avviato con Docker Compose [6]. I tre processi `mongod` ascoltano sulle porte 27017/27018/27019 e condividono la rete, così da essere raggiungibili come `localhost:2701x` sia tra container sia dall'host. La connection string in `.env` elenca tutti e tre i membri:

Listing 1: Connection string del replica set.

```
mongodb://localhost:27017,localhost:27018,localhost:27019/?replicaSet=rs0
```

Il driver PyMongo scopre da solo la topologia e indirizza le operazioni al nodo giusto. Interrogando `replSetGetStatus` vediamo che sono stati correttamente creati 3 container Docker (ovviamente in locale) in ascolto su porte diverse. Notiamo anche che uno è stato eletto PRIMARY, e sarà il nodo che riceverà tutte le scritture, mentre gli altri due sono di default diventati SECONDARY, che replicheranno in modo asincrono a meno che le richieste non siano specifiche su nodi secondary (cosa che sfrutterò nella RQ4).

Ingestione. Importo il CSV piatto nella collection `workouts`, una riga per esercizio, a batch di 5.000 documenti via `insert_many`. Due accortezze in fase di import: (i) le colonne `level` e `goal` contengono liste serializzate come stringhe (es. `"['Beginner', 'Novice']"`), le riconverto in veri array, che BSON memorizza come array nativi, fondamentale per poterci fare `$unwind` nella RQ2; (ii) le date diventano `datetime` e i pochi valori mancanti diventano `None`, cioè BSON `null`. Su un replica set il write concern di default è `w:"majority"`: ogni batch è confermato solo quando la maggioranza dei nodi (2 su 3) ha applicato la scrittura, quindi la durabilità è garantita anche se un nodo cade subito dopo l'import. La collection `workouts` riproduce fedelmente il CSV: il punto di partenza denormalizzato, volutamente inefficiente, su cui costruire RQ1.

L'ambiente software è Python 3.11+ con `pymongo`, `pandas`, `python-dotenv`, `matplotlib` e `seaborn`; le misure di tempo e di storage sono ottenute con gli strumenti diagnostici nativi di MongoDB (`collStats`, `explain`, `replSetGetStatus`). I valori numerici riportati sono quelli osservati su una singola macchina con i tre nodi in locale: contano gli ordini di grandezza e i rapporti relativi, non i millisecondi assoluti.

3 RQ1 — Modellazione: dal CSV piatto ai documenti annidati

Nel CSV ogni esercizio ripete tutti i metadati del suo programma: per un programma con 230 esercizi, titolo, descrizione, livello e obiettivo sono duplicati 230 volte. In un database relazionale normalizzerei in quattro tabelle (`programs`, `weeks`, `days`, `exercises`) e ricostruirei il programma con tre join. MongoDB offre una terza via, l'embedding: un documento per programma, con la gerarchia settimane → giorni → esercizi annidata dentro.

Avrei potuto scegliere, invece dell'embedding, il referencing (documenti separati con riferimenti, lo stile "relazionale" di MongoDB), ma dato che quando si parla di piani di allenamento il pattern di accesso è per l'intero piano e non per i sotto-oggetti, ha più senso proseguire con l'embedding. Dunque costruisco la collection `programs`, senza far uscire i dati dal database, con una pipeline che aggrega giorno → settimana → programma e termina in `$out`:

Listing 2: Schema della pipeline di rimodellazione (semplificato).

```
[ {$group: {_id:{prog,week,day}, exercises:{$push:...}}}, // giorno
  {$group: {_id:{prog,week}, days:{$push:...}}}, // settimana
  {$group: {_id:"$prog", weeks:{$push:...}}}, // programma
  {$addFields: {weeks: {$sortBy: ...}}}, // ordino settimane e giorni
  {$out: "programs"} ] // allowDiskUse=True
```

Notiamo come dai 605.033 documenti di partenza sia riuscito a collassarli in 2.598 documenti unici, dato che il titolo fa da chiave naturale (`_id`) e quindi l'unicità è garantita dal database. Ho impostato `allowDiskUse=True` in quanto, data la mole di documenti, i `$group` superano il limite di default di MongoDB di 100 MB di RAM per stage: in questo modo permettiamo lo spilling su disco. Inoltre, dato che `$push` non garantisce l'ordine degli elementi, ho aggiunto uno stage finale con `$sortBy` che riordina settimane e giorni prima del `$out`.

Ogni singolo documento della collezione risultante è costituito come segue:

- Programma (id univoco che fa da chiave);
- Equipaggiamento;
- Livello di esperienza;
- Durata in settimane;
- Giorni per settimana;
- Esercizi per giorno (ognuno con i suoi campi: name, sets, reps, intensity).

Per esempio il programma `Weightlifting Mobility program` diventa un singolo documento con `equipment`, livelli, 7 settimane, ognuna con i suoi giorni e dentro ogni giorno i suoi esercizi. Nessuna ricostruzione applicativa, nessun join.

Tabella 1: Confronto tra rappresentazione piatta (**workouts**) e annidata (**programs**).

Metrica	Piatta (workouts)	Annidata (programs)	Variazione
Documenti	605.033	2.598	−99.6%
Dati logici	413,8 MB	54,4 MB	−87%
Storage su disco	45,3 MB	7,4 MB	−84%
Dim. media documento	0,7 KB	~21 KB	—
Lettura “programma X”	~165 ms (325 doc)	~1,2 ms (1 doc)	~ 100×

Il confronto (Tabella 1) quantifica i due vantaggi dell’embedding su questo dataset:

1. Storage: ho ridotto notevolmente i dati memorizzati collassando i documenti, al costo di aumentare la dimensione del documento singolo, che resta comunque molto sotto la soglia dei 16 MB massimi previsti per singolo documento (il vincolo strutturale da verificare sempre prima di scegliere l’embedding).
2. Pattern di accesso: l’intero programma si recupera con una lettura di un singolo documento (~1,2 ms, servita dall’indice di default su `_id`); nella versione piatta servono 325 documenti da filtrare, e senza indice questo comporta una scansione completa dei 605k documenti (~165 ms), da riassemblare poi lato applicazione. L’accesso al documento è inoltre atomico: una modifica al programma intero non richiede transazioni multi-documento, in quanto i dati sono stati raggruppati.

Quando invece conviene il referencing? Se i sotto-oggetti fossero acceduti indipendentemente dal padre (es. “tutti i giorni di allenamento di oggi, di qualunque programma”) allora valuterei uno schema referencing. È il classico trade-off del modello documentale: si modella in base a come si legge, non in base alla struttura “pura” dei dati.

Risposta a RQ1. Sì: per questo dataset, con questo pattern di accesso, il modello annidato è superiore su storage, semplicità e atomicità.

4 RQ2 — Aggregation framework: analytics nel database

Inizio dalla domanda più naturale: quali sono gli esercizi più usati nei programmi? Dal momento in cui ho compattato i documenti, per rispondere è necessario utilizzare la soluzione proposta da MongoDB, l’aggregation framework [5]: una pipeline di stage che unisce la clausola `GROUP BY` (alla quale siamo già abituati) a funzioni analitiche quali `$unwind`, `$group`, `$sort`, `$facet`, ...La clausola chiave qui è `$unwind`, già vista anche in Neo4j, che permette di “srotolare” un array producendo un documento per elemento.

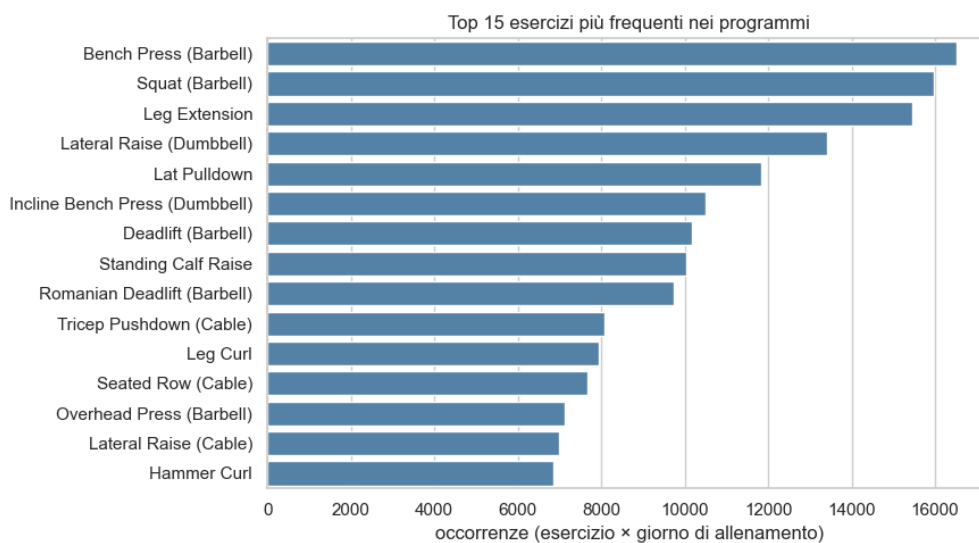


Figura 1: Esercizi più frequenti nei programmi, ottenuti con tre `$unwind` seguiti da `$group/$sort`.

La successione dei tre `$unwind` ha srotolato i documenti collassati, permettendo di tornare alle 605 mila righe di partenza; la clausola `$group` ha poi permesso di contarle per nome (Figura 1). Vediamo che i primi esercizi

sono esercizi multiarticolari (panca piana, squat, stacco da terra), chiamati anche “fondamentali” in quanto coinvolgono molti muscoli del corpo in un solo movimento e sono usati nella maggior parte dei programmi di forza. Il risultato è coerente con quanto ci si poteva aspettare.

Sfruttando nuovamente la clausola `$unwind` è possibile plottare la distribuzione dei programmi per `level` e `goal` (Figura 2). Notiamo che le somme superano i 2.598 documenti collassati, ma è normale, in quanto un piano indica più livelli e più obiettivi. Osserviamo che il dataset non è orientato agli estremi: Beginner e Advanced hanno molti meno piani rispetto a Novice e Intermediate. Gli obiettivi più presenti invece risultano essere Bodybuilding (1723), Muscle & Sculpting (936) e Powerbuilding (905), il che conferma perché gli esercizi più presenti nei vari piani fossero i fondamentali.

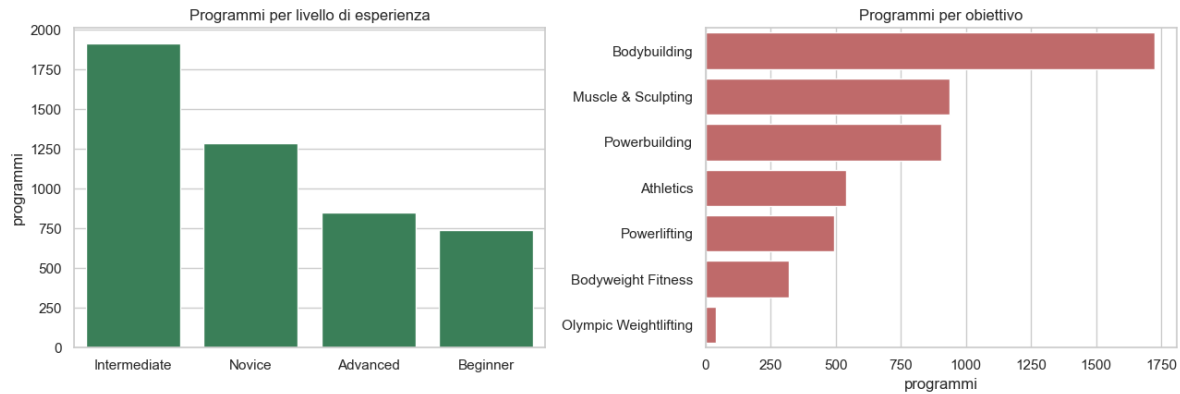


Figura 2: Distribuzione dei programmi per livello (sinistra) e obiettivo (destra), ottenuta con `$unwind` sugli array nativi `level` e `goal`.

Con la query successiva ho stampato la distribuzione delle strutture richieste per eseguire i programmi (Figura 3). Questa query è stata fatta per dimostrare che, anche usando una tecnica di embedding e quindi collassando i documenti, possiamo usare funzioni quali `$map` e `$sum` al posto di `$unwind` per andare a lavorare dentro gli array: è così che calcolo le statistiche sugli esercizi per programma (media 232,9, min 1, max 5.040). Questo dimostra il potenziale e la versatilità che vi è dietro le aggregation expression di MongoDB. Analizzando il plot vediamo come la maggior parte dei piani richieda una Full Gym o almeno una Garage Gym per essere eseguita.

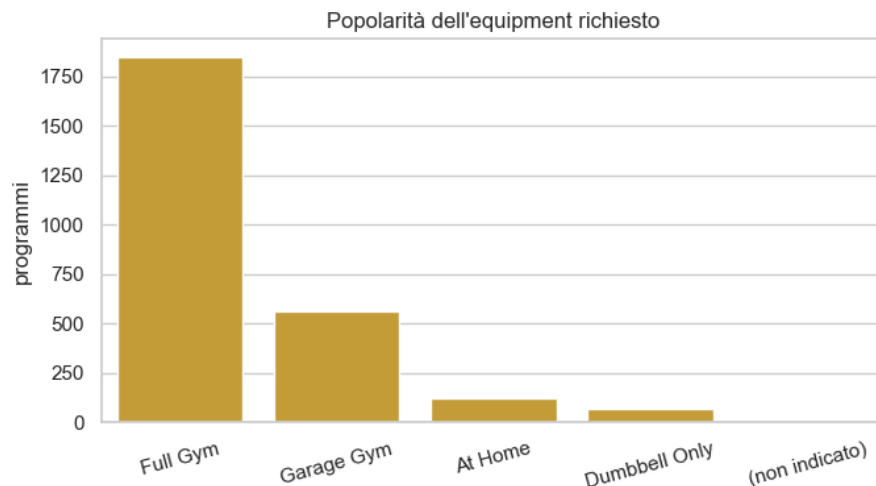


Figura 3: Equipment richiesto dai programmi. I programmi a corpo libero o ad attrezzatura minima sono una nicchia.

Infine ho voluto approfondire un aspetto molto utile di MongoDB, il `$facet`. Questa clausola permette di restituire un unico documento in output basato su tutti i risultati, eseguendo più sotto-pipeline indipendenti in una sola scansione del DB: in una passata ottengo i 2.598 programmi totali, la durata media (8,8 settimane, 69 minuti per workout), i top obiettivi e la distribuzione per livello. Tipicamente questa tecnica viene usata in pagine di riepilogo e/o dashboard, in quanto con una sola round-trip al database si ottengono N metriche. Questo la rende molto simile alla funzione `.describe()` di pandas, che in un sol colpo restituisce varie metriche del dataframe.

Risposta a RQ2. L'aggregation framework mi ha permesso di fare varie e vaste operazioni sui dati del mio database pur mantenendo alta l'efficienza, senza spostare i dati fuori dal database; a Python restano solo i grafici.

5 RQ3 — Indici e performance

Quando lavoriamo con database, e soprattutto con grandi quantità di dati, è necessario saper sfruttare gli indici messi a disposizione. Così come fatto nel corso di Database e per Neo4j, ho deciso di implementare una sezione dedicata agli indici anche in questo progetto.

Su MongoDB, se non vi sono indici sui campi dei documenti, quello che avviene è una **COLLSCAN**: viene letta tutta la collection e il filtro è applicato documento per documento. Lavoro sulla collection `workouts` piatta, non collasata, in quanto rappresenta il caso peggiore (605k documenti). Lancio due query realistiche, la prima filtrando sul singolo campo `exercise_name` e la seconda applicando un doppio filtro (`equipment + exercise_name`), e profilo tutto usando `explain("executionStats")` che, come `PROFILE` su Neo4j, permette di vedere il piano scelto, il numero di documenti esaminati, il tempo di esecuzione della query, ecc.

Come previsto, senza alcun indice la scansione **COLLSCAN** avviene su tutta la collezione (605.033 documenti esaminati, ~280 ms). Ragionando in termini di dataset molto più grandi, come quelli utilizzati dalle grandi aziende, query di questo tipo non sono accettabili: con l'aumentare dei dati aumenta anche il tempo richiesto per eseguire la query. Creo dunque un indice su `exercise_name` e un indice composto su `equipment + exercise_name` e ripeto le stesse query profilandole.

Tabella 2: Effetto degli indici, misurato con `explain("executionStats")` su `workouts`.

Query	Piano	docsExamined	nReturned	Tempo
1 campo, no indice	COLLSCAN	605.033	16.508	~280 ms
1 campo, indice singolo	IXSCAN+FETCH	16.508	16.508	~40 ms
2 campi, indice composto	IXSCAN+FETCH	4.116	4.116	~6 ms
covered query	IXSCAN (no FETCH)	0	4.116	~2 ms

La Tabella 2 riassume i risultati:

1. **IXSCAN + FETCH**: con l'indice MongoDB naviga il B-tree fino alle chiavi che soddisfano il filtro (`keysExamined = nReturned`) e va a prendere solo quei documenti: `docsExamined` crolla da 605.033 a 16.508 e il tempo da ~280 a ~40 ms.
2. Indice composto: la query sui due campi utilizza l'indice composto e riporta i 4.116 documenti in ~6 ms. L'ordine dei campi conta (prefix rule): lo stesso indice servirebbe anche query sul solo `equipment`, ma non sul solo `exercise_name`.
3. Covered query: caso particolare di query in cui vengono richiesti solo i campi che sono nell'indice. La pipeline eseguita da MongoDB passa da “lettura indice, fetch dei documenti, ritorno dei risultati” a “lettura indice e ritorno dei risultati”: `docsExamined = 0`, lo stage `FETCH` sparisce e il tempo viene ulteriormente abbattuto, in quanto si salta la fase più dispendiosa nell'esecuzione di una query, ovvero leggere effettivamente i documenti.

Nonostante la diminuzione significativa del tempo richiesto per recuperare i dati, creare indici comporta aggiungere strutture dati che vanno memorizzate (e che quindi occupano spazio) e che devono essere aggiornate a ogni scrittura. Su collezioni nelle quali vengono eseguite frequenti scritture potrebbe essere necessario calcolarne i benefici, per capire l'effettivo guadagno.

Risposta a RQ3. L'impatto è di 2–3 ordini di grandezza sui documenti esaminati; `explain` rende la decisione misurabile invece che intuitiva.

6 RQ4 — Il teorema CAP, sperimentato sul replica set

Il teorema CAP [2, 3] dice che un sistema distribuito, di fronte a una partizione di rete (P), deve scegliere tra essere consistente (C) e disponibile (A). MongoDB sceglie la consistenza: è un sistema CP. In questa sezione non mi limito a dichiararlo, ma dimostro la reazione del sistema di fronte a “guasti” sui nodi.

Su MongoDB possiamo impostare il write concern, ovvero la policy secondo la quale viene descritta la durabilità di una scrittura, ai seguenti livelli:

- `w:1` (scrittura ok dopo il nodo primary);

- **w:2** (scrittura ok dopo primary + 1 secondary);
- **w:"majority"** (scrittura ok dopo che ha scritto sulla maggioranza dei nodi).

È possibile anche impostare la read preference, ovvero scegliere da quali nodi leggere, ai seguenti valori:

- **primary** (leggiamo dal nodo primario, il default);
- **secondary** (leggiamo dai nodi secondari);
- **nearest** (leggiamo dal nodo più vicino nella rete distribuita);
- **primaryPreferred** (preferiamo il primario, ma se è down leggiamo da un secondary).

Per simulare la partizione fermerò il container Docker del primary e osserveremo cosa succede.

6.1 — Write concern: quanto costa la durabilità. Misuro il costo della garanzia inserendo gli stessi batch (500 documenti $\times$ 5 run) con i due livelli estremi: **w:"majority"** paga un “sovrapprezzo” rispetto a **w:1** (20,1 contro 15,6 ms medi per batch), perché la conferma deve attendere la replica su almeno un secondary. Su localhost il divario è di pochi millisecondi, ma in un cluster geo-distribuito diventa la latenza di rete tra datacenter. In cambio **majority** garantisce che la scrittura sopravviva a un failover, in quanto una scrittura confermata solo con **w:1** viene annullata (rollback) se il primary cade prima di replicarla. Questo rappresenta proprio il trade-off tra C e A: più aggiungiamo garanzie di consistenza, più aumenta la latenza.

6.2 — Read preference: leggere dai secondary. Con **readPreference=secondary** il driver instrada la lettura su un nodo secondario: nell’esperimento la stessa query viene servita dalla porta 27017 (primary) o 27018 (secondary) a seconda della preference, e il count sui due nodi coincide (605.033 documenti, la replica è al passo). Il prezzo è la consistenza eventuale: i secondary applicano l’oplog in modo asincrono, quindi una lettura può restituire dati stale di qualche istante.

6.3 — Failover: il cluster perde il primary. L’esperimento centrale della RQ4. Uccido il container del primary con **docker kill** e, una volta al secondo, provo a scrivere (timeout 2 s) e a leggere da un secondary, registrando gli esiti su una timeline.

Perché **kill** e non **stop**? Con **docker stop** il mongod riceve un SIGTERM e fa uno stepdown graceful: avvisa i secondary, che eleggono il nuovo primary quasi istantaneamente (l’ho verificato, e la finestra di indisponibilità è praticamente invisibile). È il meccanismo usato nei riavvii pianificati. Il teorema CAP però parla di guasti non pianificati: usando **docker kill** mi assicuro che il segnale sia un SIGKILL, e questo fa partire il timeout di default di 10 s (**electionTimeoutMillis**) prima che i secondary, accortisi della scomparsa del primary tramite gli heartbeat falliti, indicano l’elezione [7].

La timeline mostra il comportamento CP di MongoDB in diretta: nella mia run i primi tre tentativi di scrittura (a $t = 0, 3$ e 6 s) falliscono, e a $t \approx 9$ s la scrittura torna OK sul nuovo primary (il nodo sulla porta 27018).

- Le scritture si bloccano per l’intera durata dell’elezione (~ 10 s): il driver non trova un primary (**ServerSelectionTimeoutError**) perché al momento non esiste. Il sistema sacrifica la disponibilità in scrittura pur di non rischiare due primary concorrenti (split-brain).
- Le letture dai secondary continuano per tutto il tempo: la disponibilità in lettura non cambia.
- I due nodi superstiti formano ancora la maggioranza (2 su 3), quindi possono eleggere un nuovo primary e il servizio riprende da solo. Se avessi fermato due nodi, il nodo superstite sarebbe rimasto secondary per sempre: nessuna maggioranza, nessun primary, niente scritture.

Il vecchio primary, riavviato, rientra nel replica set come SECONDARY: si risincronizza dal nuovo primary e recupera le scritture perse durante la sua assenza. Nessun intervento manuale.

Tabella 3: Mappatura degli eventi osservati sul teorema CAP.

Evento osservato	Lettura CAP
Scritture bloccate durante l’elezione (~ 10 s)	sotto partizione MongoDB sacrifica A per C $\rightarrow$ sistema CP
w:"majority" più lento di w:1	il grado di C si paga in latenza, scrittura per scrittura
Serve la maggioranza per eleggere il primary	protezione dallo split-brain: mai due “verità” divergenti

Risposta a RQ4. La mappa completa è in Tabella 3: CAP per MongoDB non è un’etichetta statica, ma una manopola. Write concern e read preference permettono di scegliere, per singola operazione, quanto pagare in latenza e disponibilità per la consistenza.

7 Conclusioni

In questo approfondimento ho esplorato MongoDB sui quattro assi delle research question, con esperimenti riproducibili su un replica set Docker a 3 nodi:

- RQ1 — Modellazione. Il dataset piatto (605k righe con metadati duplicati) è stato rimodellato in 2.598 documenti annidati programma → settimane → giorni → esercizi con una sola pipeline di aggregazione (`$group × 3 + $out`). Risultato: dati logici da 414 a 54 MB (−87%) e l’intero programma recuperabile con una singola lettura atomica (~1,2 ms contro ~165). L’embedding vince quando l’unità di accesso coincide con il documento; il referencing resta preferibile per sotto-oggetti condivisi o array a crescita illimitata (limite BSON 16 MB).
- RQ2 — Aggregation framework. Tutta l’analisi descrittiva (top esercizi, distribuzioni per livello/obiettivo/equipment, statistiche multi-metrica con `$facet`) è stata eseguita dentro il database: `$unwind` riapre la gerarchia quando serve la granularità fine, `$map/$size` navigano gli array senza esploderli.
- RQ3 — Indici. Con `explain("executionStats")` ho misurato il passaggio da COLLSCAN (605k documenti esaminati) a IXSCAN (solo i documenti restituiti) fino alla covered query (`docsExamined = 0`): 2–3 ordini di grandezza, al costo di spazio e overhead in scrittura.
- RQ4 — CAP. Il failover dal vivo ha mostrato il carattere CP di MongoDB: alla caduta del primary le scritture si bloccano per i ~10 secondi dell’elezione (consistenza pagata in disponibilità), le letture dai secondary proseguono in consistenza eventuale, e write concern e read preference permettono di regolare il trade-off per singola operazione.

Il filo conduttore è che le scelte di design di MongoDB (documenti, pipeline, indici, replica) diventano comprensibili solo guardando cosa ottimizzano: il modello documentale ottimizza il pattern di accesso, l’aggregation evita di muovere i dati, gli indici comprano letture con scritture, il replica set compra consistenza con disponibilità. Strumenti come `explain`, `collStats` e `replSetGetStatus` rendono ognuno di questi trade-off misurabile, ed è questo che li rende decisioni ingegneristiche e non atti di fede. La sperimentazione completa, con istruzioni di installazione e uso, è pubblicata sul repository indicato in nota al titolo.

Riferimenti bibliografici

- [1] Bradshaw, S., Brazil, E., and Chodorow, K. (2019). MongoDB: The Definitive Guide. O'Reilly Media, 3rd edition.
- [2] Brewer, E. (2012). CAP twelve years later: How the “rules” have changed. *Computer*, 45(2):23–29.
- [3] Gilbert, S. and Lynch, N. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59.
- [4] Kaggle (2023). Fitness exercises dataset (bootcamp training programs). [https://www.kaggle.com/](https://www.kaggle.com/.). Dataset: `programs_detailed_bootcamp_kaggle.csv`. Accessed: 2026-06-11.
- [5] MongoDB, Inc. (2024a). Aggregation pipeline — MongoDB manual. <https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>. Accessed: 2026-06-11.
- [6] MongoDB, Inc. (2024b). Replication — MongoDB manual. <https://www.mongodb.com/docs/manual/replication/>. Accessed: 2026-06-11.
- [7] Ongaro, D. and Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *Proc. 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, pages 305–319.